PYTHON FOR DATA · 40 MIN

Introduction to Machine Learning

Learning from data with scikit-learn · Unit 4

Data

Features and target

`X  y`

Split

Train and test

`train_test_split`

Model

Fit and predict

`fit  predict`

Evaluate

Score and validate

`score  metrics`

The Machine Learning Workflow

~2 min

Learn from data, then test on new data.

1 Data

Features X and a target y.

`load_iris` `X` `y`

2 Split

Keep a test set out of reach.

`train_test_split`

3 Fit and score

Train, predict, then measure.

`fit` `predict` `score`

4 Improve

Cross-validate and use pipelines.

`cross_val_score` `pipeline`

Supervised learning has a target. Unsupervised learning does not.

Features and Target

~4 min

Rows are samples. Columns are features.

```
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
X, y = iris.data, iris.target
print(X.shape, y.shape)
print(X.columns.tolist()[:2])
print(iris.target_names.tolist())
```

Output

```
(150, 4) (150,)
['sepal length (cm)', 'sepal width (cm)']
['setosa', 'versicolor', 'virginica']
```

Features X

The input columns. Each row is one sample, here one flower.

Target y

The answer to predict. A labelled target means supervised learning.

Train and Test Split

~4 min

Hold out data the model never sees.

```
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y)
print(X_tr.shape, X_te.shape)
print(y_te.value_counts().tolist())
```

Output

```
(120, 4) (30, 4)
[10, 10, 10]
```

test_size

0.2 keeps 20% of the rows for testing. The model never trains on them.

random_state, stratify

A fixed seed repeats the split. stratify keeps class shares equal.

Fit and Predict

~4 min

Learn from the training rows only.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_tr, y_tr)
pred = model.predict(X_te)
print(pred[:6].tolist())
print(y_te.tolist()[:6])
```

Output

```
[0, 1, 0, 2, 0, 1]
[0, 1, 0, 2, 0, 1]
```

fit

Learns the pattern from X_tr and y_tr. Never call it on test data.

predict

Returns a label for each row of new data. Compare it with y_te.

Evaluate the Model

~4 min

Score on data held out for testing.

```
from sklearn import metrics
print(round(metrics.accuracy_score(y_te, pred), 3))
print(round(model.score(X_te, y_te), 3))
print(metrics.confusion_matrix(y_te, pred))
```

Output

```
1.0
1.0
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]
```

accuracy

Share of correct predictions. 30 test rows is small, so stay cautious.

Confusion matrix

Rows are true classes, columns are predictions. Off-diagonals are errors.

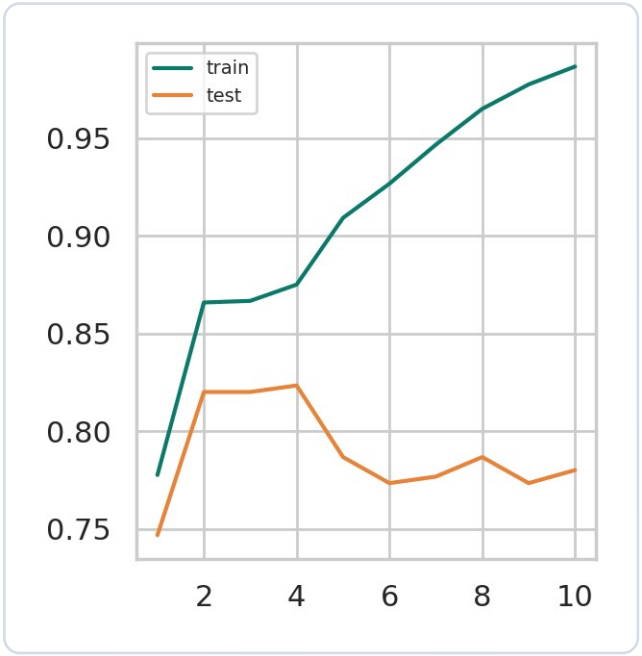
Overfitting

~4 min

A flexible model can memorize noise.

```
from sklearn import datasets, model_selection, tree
import matplotlib.pyplot as plt
X, y = datasets.make_moons(300, noise=0.4,
                           random_state=1)
clf, ds = tree.DecisionTreeClassifier(), range(1, 11)
tr, te = model_selection.validation_curve(clf, X, y,
    param_name="max_depth", param_range=ds, cv=5)
plt.plot(ds, tr.mean(1), ds, te.mean(1))
plt.legend(["train", "test"])
```

Plot



Overfitting

Train score keeps rising while test score drops.

The fix

Limit depth or add data. Check with cross validation.

Cross-Validation

~4 min

Test on several splits, not one.

```
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
scores = cross_val_score(
    DecisionTreeClassifier(random_state=0),
    X, y, cv=5)
print(scores.round(2).tolist())
print(round(scores.mean(), 3))
```

Output

```
[0.97, 0.97, 0.9, 0.97, 1.0]
0.96
```

cv=5

Splits the data into 5 folds. Each fold is the test set once.

Mean and spread

The average is steadier than one split. Check the spread too.

Regression Basics

~4 min

Predict a number instead of a class.

```
import numpy as np
from sklearn.linear_model import LinearRegression
rng = np.random.default_rng(0)
xs = rng.uniform(0, 10, (50, 1))
ys = 3 * xs[:, 0] + 2 + rng.normal(0, 1, 50)
reg = LinearRegression().fit(xs, ys)
print(reg.coef_.round(2), round(reg.intercept_, 2))
print(round(reg.score(xs, ys), 3))
```

Output

```
[3.05] 1.76
0.988
```

coef_, intercept_

The learned slope and offset. Here close to 3 and 2.

R squared

score gives R squared. Use test data for a fair value.

Pipelines and Scaling

~4 min

Chain steps so they learn from train only.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe = make_pipeline(StandardScaler(),
    LogisticRegression(max_iter=200))
pipe.fit(X_tr, y_tr)
print(round(pipe.score(X_te, y_te), 3))
print(pipe.steps[0][0], pipe.steps[1][0])
```

Output

```
0.967
standardscaler logisticregression
```

Scale features

StandardScaler centres each column and sets its spread to 1.

Why a pipeline

The scaler is fitted on training rows only, so nothing leaks.

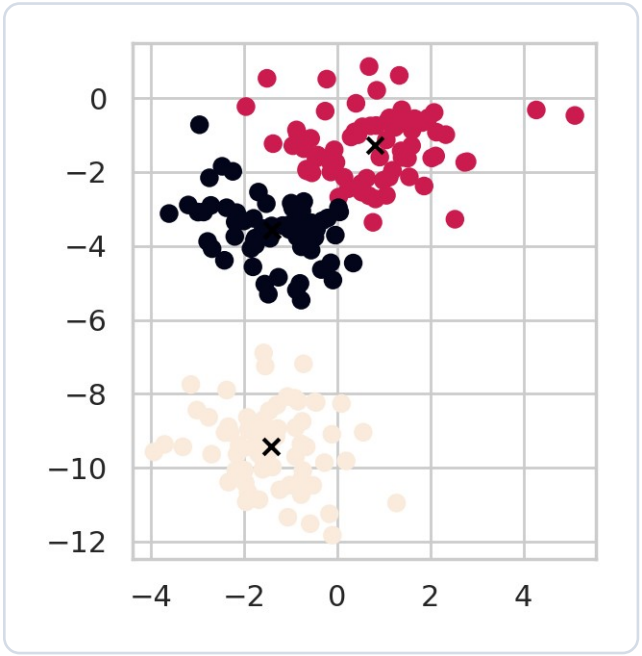
Unsupervised: KMeans

~4 min

Find groups without a target.

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
X, _ = make_blobs(200, centers=3, random_state=2)
km = KMeans(n_clusters=3, n_init=10, random_state=0)
labels = km.fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(*km.cluster_centers_.T, c="k", marker="x")
```

Plot



No target y

KMeans only sees X. It groups rows that sit close together.

Choose k

You pick n_clusters. Try several values and compare.

Machine Learning Cheat Sheet

~2 min

Match the task to the tool.

Task	Tool
Features and target	`X, y`
Hold out test data	`train_test_split`
Train a model	`model.fit(X_tr, y_tr)`
Predict labels	`model.predict(X_te)`
Score a model	`model.score, metrics.accuracy_score`
Fair estimate	`cross_val_score(model, X, y, cv=5)`
Scale, then model	`make_pipeline(StandardScaler(), model)`
Find groups	`KMeans(n_clusters=k).fit_predict(X)`

Never fit on test data. Compare train and test scores to spot overfitting.

PYTHON FOR DATA · 40 MIN

Types of Learning

Supervised, unsupervised and reinforcement learning · Unit 4

Supervised
Learn from labels
`LinearRegression`

Unsupervised
Find hidden groups
`KMeans`

Reinforcement
Learn from rewards
`Q-learning`

Choosing
Match task to type
`compare`

~3 min

Three Ways a Machine Learns

What the data looks like decides the type.

1

Supervised

Learn from examples that have answers.

`labels`

2

Unsupervised

Find patterns in data with no answers.

`no labels`

3

Reinforcement

Learn by trying actions and getting rewards.

`rewards`

Ask first: do I have answers, no answers, or a reward after each action?

~3 min

Comparing the Three Types

Data, goal and a familiar example.

Type	Data	Goal	Example
Supervised	Labeled	Predict a label	Spam filter
Unsupervised	No labels	Find structure	Customer groups
Reinforcement	Rewards	Maximize reward	Game player

Supervised and unsupervised learn from a fixed data set. Reinforcement learns by acting.

~4 min

Supervised: Regression

Predict a number from examples.

```
from sklearn.linear_model import LinearRegression
X = [[1], [2], [3], [4]]
y = [10, 20, 30, 40]
m = LinearRegression().fit(X, y)
print(m.predict([[5]]))
print(round(m.coef_[0], 1), round(m.intercept_, 1))
```

Output

```
[50.]
10.0 0.0
```

Inputs and answers

X holds the inputs. y holds the known answers the model learns from.

Regression

Predicts a number. Each step adds 10, so an input of 5 gives 50.

~4 min

Supervised: Classification

Predict a category from examples.

```
from sklearn.neighbors import KNeighborsClassifier
X = [[1, 1], [2, 1], [8, 9], [9, 8]]
y = ["cat", "cat", "dog", "dog"]
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X, y)
print(knn.predict([[2, 2], [8, 8]]).tolist())
```

Output

```
['cat', 'dog']
```

Classification

Predicts a label such as cat or dog instead of a number.

Nearest neighbour

Each new point takes the label of the closest training point.

~5 min

Supervised: Test the Model

Score it on data it has not seen.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, random_state=0)
print(Xtr.shape, Xte.shape)
tree = DecisionTreeClassifier(random_state=0)
tree.fit(Xtr, ytr)
print(round(tree.score(Xte, yte), 2))
```

Output

```
(105, 4) (45, 4)
0.98
```

Train and test

30 percent of the rows are kept back. The model never sees them while learning.

score

For a classifier, score is accuracy: the share of test rows predicted correctly.

~4 min

Unsupervised: Clustering

Group similar points without labels.

```
from sklearn.cluster import KMeans
points = [[1, 1], [1, 2], [8, 8], [9, 8]]
km = KMeans(n_clusters=2, n_init=10,
            random_state=0).fit(points)
print(sorted(km.cluster_centers_.round(1).tolist()))
lab = km.labels_
print(lab[0] == lab[1], lab[2] == lab[3])
```

Output

```
[[1.0, 1.5], [8.5, 8.0]]
True True
```

No labels given

We only choose n_clusters=2. KMeans finds the two groups itself.

Centers

Each group has a center. The first two points share one, the last two share the other.

~4 min

Unsupervised: PCA

Shrink many columns into a few.

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
X, _ = load_iris(return_X_y=True)
pca = PCA(n_components=2).fit(X)
print(pca.transform(X).shape)
ratio = pca.explained_variance_ratio_
print(ratio.round(2).tolist())
```

Output

```
(150, 2)
[0.92, 0.05]
```

Fewer columns

The 4 columns become 2, so the data is easier to plot and store.

Variance kept

The two new columns keep 0.92 and 0.05 of the variation, about 98 percent.

~3 min

Reinforcement: Key Ideas

An agent learns by acting in an environment.

1

Agent

The learner that chooses actions.

`robot` `player`

2

Environment

The world that responds to actions.

`game` `road`

3

Reward

A score for each action taken.

`+1` `-1`

4

Policy

The rule for choosing the next action.

`best action`

The agent keeps the policy that earns the most reward over time.

~5 min

Reinforcement: The Update

Q-learning scores each action.

```
def update(Q, s, a, r, n):
    target = r + 0.9 * max(Q[n])
    Q[s][a] += 0.5 * (target - Q[s][a])
Q = [[0.0, 0.0] for _ in range(4)]
update(Q, 2, 1, 1, 3)
print(Q[2])
```

Output

```
[0.0, 0.5]
```

Q table

Q[s][a] is the score of action a in state s. All scores start at 0.

Update

Reward 1 pulls the score halfway (0.5) toward the target. 0.9 values later rewards.

~5 min

Reinforcement: Learn by Repeating

A walk to a goal in state 3.

```
Q = [[0.0, 0.0] for _ in range(4)]
for _ in range(20):
    for s in range(3):
        for a in (0, 1):
            n = max(0, s + (1 if a else -1))
            update(Q, s, a, int(n == 3), n)
print([int(q[1] > q[0]) for q in Q[:3]])
print([round(q[1], 2) for q in Q[:3]])
```

Output

```
[1, 1, 1]
[0.81, 0.9, 1.0]
```

Actions

Action 0 steps left and action 1 steps right. State 3 is the goal.

The result

Right is best in every state. Scores rise toward the goal: 0.81, 0.9, then 1.0.

~3 min

Which Type Fits?

Match the situation to a type.

Situation	Type
Predict house price from past sales	Supervised
Group shoppers by their habits	Unsupervised
Teach a robot to walk	Reinforcement
Mark mail as spam from examples	Supervised
Reduce 100 columns to 2	Unsupervised
Play a game to win	Reinforcement

Labels point to supervised. No labels point to unsupervised. Rewards point to reinforcement.

~3 min

Types of Learning Cheat Sheet

Match the task to the tool.

Task	Tool
Predict a number	`LinearRegression`
Predict a category	`KNeighborsClassifier`
Split and score	`train_test_split, score`
Find groups	`KMeans`
Reduce columns	`PCA`
Learn from rewards	`Q table, update rule`

Always judge a supervised model on test data, never on the data it trained on.

PYTHON FOR DATA · 40 MIN

Data Preparation for ML

Feature engineering, scaling, train-test split and cross-validation · Unit 4

Features
Build better inputs
`get_dummies`

Scaling
Put columns on one scale
`StandardScaler`

Split
Keep test data apart
`train_test_split`

Validation
Score more than once
`cross_val_score`

~3 min

Four Steps Before Training

Prepare the data so the model learns fairly.

1

Features

Create and encode useful columns.

`get_dummies`

2

Scaling

Bring numbers to a similar range.

`StandardScaler`

3

Split

Hold back data for a fair test.

`train_test_split`

4

Validate

Test on several different splits.

`cross_val_score`

The order matters: split first, then fit scalers on the training data only.

~4 min

Features: Dates and Ratios

Build new columns from old ones.

```
dates = ["2026-01-05", "2026-03-14"]
df = pd.DataFrame({
    "date": pd.to_datetime(dates),
    "price": [200, 300], "qty": [2, 5]})
df["month"] = df["date"].dt.month
df["weekday"] = df["date"].dt.dayofweek
df["total"] = df["price"] * df["qty"]
print(df[["month", "weekday", "total"]])
```

Output

```
   month  weekday  total
0      1        0    400
1      3        5   1500
```

Dates

dt.month and dt.dayofweek pull simple numbers out of a date. Monday is 0.

Combine columns

price times qty gives a total, which may predict better than either column.

~4 min

Features: Encode Categories

Models need numbers, not words.

```
df = pd.DataFrame({"city": ["Pune", "Goa"],
                   "size": ["S", "L"]})
sizes = {"S": 1, "M": 2, "L": 3}
df["size"] = df["size"].map(sizes)
enc = pd.get_dummies(df, columns=["city"],
                     dtype=int)
print(enc)
```

Output

```
   size  city_Goa  city_Pune
0     1         0          1
1     3         1          0
```

Ordered values

S, M and L have an order, so map them to 1, 2 and 3.

One-hot

get_dummies makes a 0 or 1 column for each city. Cities have no order.

~3 min

Features: Skewed Numbers

A log shrinks huge gaps.

```
import numpy as np
income = pd.Series([10, 100, 1000, 100000])
logged = np.log1p(income)
print(logged.round(2).tolist())
```

Output

```
[2.4, 4.62, 6.91, 11.51]
```

Squeeze the range

The raw values run from 10 to 100000. After the log they run from 2.4 to 11.5.

log1p

Computes log(1 + x), so a value of 0 is safe to use.

~4 min

Scaling: Standard and MinMax

Put every column on a similar scale.

```
from sklearn import preprocessing as pp
data = [[10], [20], [30]]
std = pp.StandardScaler().fit_transform(data)
mm = pp.MinMaxScaler().fit_transform(data)
print(std.round(2).tolist())
print(mm.tolist())
```

Output

```
[[-1.22], [0.0], [1.22]]
[[0.0], [0.5], [1.0]]
```

StandardScaler

Subtracts the mean and divides by the spread. The result is centered on 0.

MinMaxScaler

Squeezes values into 0 to 1. The smallest becomes 0, the largest becomes 1.

~4 min

Scaling: Fit on Train Only

Never let test data shape the scaler.

```
X_train = [[10], [20], [30]]
X_test = [[40]]
sc = pp.StandardScaler().fit(X_train)
print(sc.mean_, sc.scale_.round(2))
print(sc.transform(X_test).round(2).tolist())
```

Output

```
[20.] [8.16]
[[2.45]]
```

fit then transform

fit learns the mean and spread from training rows only.

Data leakage

Fitting on test rows leaks information and makes the score look too good.

~4 min

Train-Test Split

Hold back data for a fair test.

```
from sklearn.model_selection import train_test_split
X = [[i] for i in range(10)]
y = [0, 1] * 5
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
print(len(X_tr), len(X_te))
print(X_te)
```

Output

```
8 2
[[8], [1]]
```

test_size

0.2 keeps 20 percent of the rows for testing. Here 2 of 10.

random_state

Fixes the shuffle, so you get the same split every time you run it.

~4 min

Split: Keep the Balance

stratify keeps class shares equal.

```
y = [0] * 8 + [1] * 2
_, _, y_tr, y_te = train_test_split(
    X, y, test_size=0.5, stratify=y,
    random_state=0)
print(y_tr.count(1), y_te.count(1))
```

Output

```
1 1
```

Rare class

Only 2 of 10 labels are class 1. A plain split could put both in one set.

stratify=y

Each set gets the same share of each class. Here one class 1 row goes to each.

~4 min

Cross-Validation: KFold

Every row is tested exactly once.

```
from sklearn.model_selection import KFold
kf = KFold(n_splits=3)
for tr, te in kf.split(list(range(6))):
    print("train", tr.tolist(), "test", te.tolist())
```

Output

```
train [2, 3, 4, 5] test [0, 1]
train [0, 1, 4, 5] test [2, 3]
train [0, 1, 2, 3] test [4, 5]
```

Three folds

The rows are cut into 3 parts. Each part takes one turn as the test set.

Why

One lucky or unlucky split can mislead you. Several splits give a steadier picture.

~5 min

Cross-Validation Scores

cross_val_score does the loop for you.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
X, y = load_iris(return_X_y=True)
knn = KNeighborsClassifier(n_neighbors=5)
scores = cross_val_score(knn, X, y, cv=5)
print(scores.round(2).tolist())
print(round(scores.mean(), 2))
```

Output

```
[0.97, 1.0, 0.93, 0.97, 1.0]
0.97
```

cv=5

The model is trained and scored 5 times, once per fold.

Report the mean

The average, 0.97, is a better estimate than any single score.

~5 min

Scaling Inside Cross-Validation

A pipeline refits the scaler on each fold.

```
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(pp.StandardScaler(), knn)
scores = cross_val_score(pipe, X, y, cv=5)
print(round(scores.mean(), 2))
```

Output

```
0.96
```

No leakage

In every fold the scaler sees only that fold's training rows.

Same call

A pipeline works with cross_val_score just like a single model.

~3 min

Data Preparation Cheat Sheet

Match the task to the tool.

Task	Tool
Date parts	`dt.month, dt.dayofweek`
Encode categories	`map, get_dummies`
Scale columns	`StandardScaler, MinMaxScaler`
Split the data	`train_test_split(stratify=)`
Score several splits	`KFold, cross_val_score`
Avoid leakage	`make_pipeline`

Split first. Fit on training data only. Judge the model on data it has never seen.

PYTHON FOR DATA · 40 MIN

Linear Regression

Supervised learning · Unit 4

The line
Slope and intercept
`y = m x + b`

Fitting
Least squares
`LinearRegression`

Scoring
Errors and R squared
`r2_score`

More inputs
Several features
`coef_`

~3 min

Four Steps of Linear Regression

Predict a number with a straight line.

1

Data

Inputs X and the known target y.

`X, y`

2

Fit

Find the line that best matches the data.

`fit`

3

Predict

Use the line on new inputs.

`predict`

4

Evaluate

Measure how far off the predictions are.

`MAE` `R2`

Linear regression is supervised learning: the target y is known while training.

~4 min

The Line: Slope and Intercept

y = slope times x, plus intercept.

```
import numpy as np
x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11])
slope, intercept = np.polyfit(x, y, 1)
print(round(slope, 2), round(intercept, 2))
print(round(slope * 6 + intercept, 1))
```

Output

```
2.0 1.0
13.0
```

Slope

How much y grows when x rises by 1. Here every step adds 2.

Intercept

The value of y when x is 0. The line is y = 2x + 1, so x = 6 gives 13.

~4 min

Fit a Model

LinearRegression finds the line for you.

```
from sklearn.linear_model import LinearRegression
hours = [[1], [2], [3], [4], [5]]
marks = [35, 48, 55, 68, 75]
model = LinearRegression().fit(hours, marks)
w, b = model.coef_[0], model.intercept_
print(round(w, 2), round(b, 2))
print(model.predict([[6]]).round(1))
```

Output

```
10.0 26.2
[86.2]
```

coef_ and intercept_

Each extra study hour adds about 10 marks. Zero hours starts near 26.

predict

Takes new inputs and returns marks. 6 hours gives about 86.

~4 min

Errors Around the Line

Residual = actual minus predicted.

```
pred = model.predict(hours)
errors = np.array(marks) - pred
print(errors.round(1).tolist())
print(round((errors ** 2).sum(), 1))
print(abs(round(errors.sum(), 1)))
```

Output

```
[-1.2, 1.8, -1.2, 1.8, -1.2]
10.8
0.0
```

Least squares

The best line makes the sum of squared errors as small as possible. Here it is 10.8.

Balanced

The errors cancel out and add to 0. The line sits in the middle of the points.

~4 min

Scoring the Model

Three common measures.

```
from sklearn import metrics as mt
print(round(mt.mean_absolute_error(marks, pred), 2))
print(round(mt.mean_squared_error(marks, pred), 2))
print(round(mt.r2_score(marks, pred), 3))
```

Output

```
1.44
2.16
0.989
```

MAE and MSE

MAE is the average miss, 1.44 marks. MSE squares the misses, so big ones count more.

R squared

The share of variation explained. 1 is perfect, 0 is no better than the mean.

~5 min

A Fair Test

Fit on train, score on test.

```
from sklearn.model_selection import train_test_split
rng = np.random.RandomState(0)
X = rng.uniform(0, 10, (50, 1))
y = 3 * X[:, 0] + 5 + rng.normal(0, 1, 50)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=0)
m = LinearRegression().fit(X_tr, y_tr)
print(round(m.coef_[0], 2), round(m.intercept_, 2))
print(round(m.score(X_te, y_te), 3))
```

Output

```
2.98 5.04
0.976
```

Recovers the truth

The data was made with slope 3 and intercept 5. The fit finds 2.98 and 5.04.

score

For regression, score is R squared on the test rows: 0.976 here.

~4 min

Several Inputs

Multiple linear regression.

```
X = [[1, 0], [2, 1], [3, 0], [4, 1], [5, 0]]
y = [40, 55, 60, 75, 80]
m2 = LinearRegression().fit(X, y)
w = m2.coef_.round(2).tolist()
print(w, round(m2.intercept_, 2))
print(m2.predict([[6, 1]]).round(1))
```

Output

```
[10.0, 5.0] 30.0
[95.]
```

One weight per column

Column 1 is study hours and column 2 is a mock test, 0 or 1.

Reading weights

Each hour adds 10 marks. Taking the mock test adds 5, with hours held fixed.

~4 min

Outliers Pull the Line

One odd point can change everything.

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11])
y_out = y.copy()
y_out[-1] = 30
print(np.polyfit(x, y, 1).round(2).tolist())
print(np.polyfit(x, y_out, 1).round(2).tolist())
```

Output

```
[2.0, 1.0]
[5.8, -6.6]
```

Before

The clean data gives slope 2.0 and intercept 1.0.

After

Changing one y from 11 to 30 moves the slope to 5.8. Check for outliers first.

~3 min

Assumptions to Check

When a straight line is a fair model.

Assumption	Meaning	Quick check
Linearity	y changes in a straight line with x	Scatter plot
Independence	Rows do not depend on each other	How data was collected
Constant spread	Errors have similar size everywhere	Residual plot
Normal errors	Errors are roughly bell shaped	Histogram of residuals

If a check fails, try new features, a transform, or a different model.

~3 min

Linear Regression Cheat Sheet

Match the task to the tool.

Task	Tool
Fit a line	`LinearRegression().fit(X, y)`
Read the weights	`coef_, intercept_`
Predict new values	`predict(X_new)`
Average miss	`mean_absolute_error`
Share explained	`r2_score, model.score`
Fair test	`train_test_split`

Score on test data, and always look for outliers before trusting the line.

PYTHON FOR DATA · 40 MIN

Logistic Regression

Supervised learning for yes or no answers · Unit 4

Sigmoid
Numbers to 0 and 1

`1 / (1 + e^-z)`

Fitting
Predict a class

`LogisticRegression`

Metrics
Right and wrong calls

`confusion_matrix`

Multiclass
More than two classes

`predict_proba`

~3 min

Four Steps of Logistic Regression

Predict a class, such as pass or fail.

1

Data

Inputs X and a label y of 0 or 1.

`X, y`

2

Fit

Learn how the inputs change the odds.

`fit`

3

Predict

Get a probability, then pick a class.

`predict_proba`

4

Evaluate

Count right and wrong calls.

`confusion_matrix`

Despite the name, logistic regression is used for classification.

~4 min

The Sigmoid Curve

It squeezes any number into 0 to 1.

```
import numpy as np
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
for z in (-4, 0, 4):
    print(z, round(sigmoid(z), 3))
```

Output

```
-4 0.018
0 0.5
4 0.982
```

Probability

The output always sits between 0 and 1, so it can be read as a probability.

Middle point

z = 0 gives exactly 0.5. Large negative z gives near 0, large positive z gives near 1.

~4 min

Fit a Classifier

Hours studied, pass or fail.

```
from sklearn.linear_model import LogisticRegression
hours = [[1], [2], [3], [4], [5], [6]]
passed = [0, 0, 0, 1, 1, 1]
clf = LogisticRegression().fit(hours, passed)
print(clf.predict([[2], [5]]).tolist())
print(clf.predict_proba([[4]]).round(2).tolist())
```

Output

```
[0, 1]
[[0.36, 0.64]]
```

predict

Returns the class. 2 hours gives 0 (fail) and 5 hours gives 1 (pass).

predict_proba

Returns two probabilities: class 0 first, class 1 second. At 4 hours, 0.64 for pass.

~4 min

Probability and Threshold

Turn a probability into a decision.

```
p = clf.predict_proba([[3], [4]])[:, 1]
print(p.round(2).tolist())
print((p >= 0.5).astype(int).tolist())
print((p >= 0.7).astype(int).tolist())
```

Output

```
[0.36, 0.64]
[0, 1]
[0, 0]
```

Default 0.5

predict uses 0.5. A probability of 0.64 or more becomes class 1.

Your own threshold

Raise it to 0.7 to be stricter. Now neither student is predicted to pass.

~4 min

Reading the Weights

Coefficients are on the log-odds scale.

```
print(round(clf.coef_[0][0], 2),
      round(clf.intercept_[0], 2))
print(round(np.exp(clf.coef_[0][0]), 2))
```

Output

```
1.12 -3.92
3.07
```

coef_ and intercept_

Each extra hour adds 1.12 to the log-odds of passing. The intercept is about -3.9.

Odds ratio

np.exp turns the weight into a ratio: each extra hour multiplies the odds by about 3.

~4 min

Confusion Matrix

Count the right and wrong calls.

```
from sklearn import metrics as mt
actual = [1, 1, 1, 1, 1, 0, 0, 0]
pred = [1, 1, 1, 0, 0, 0, 0, 1]
print(mt.confusion_matrix(actual, pred).tolist())
print(mt.accuracy_score(actual, pred))
```

Output

```
[[2, 1], [2, 3]]
0.625
```

Reading the matrix

Rows are actual, columns are predicted. Top left 2 true negatives, bottom right 3 true positives.

Accuracy

5 of 8 calls were right, so accuracy is 0.625.

~4 min

Precision, Recall and F1

Accuracy is not the whole story.

```
print(mt.precision_score(actual, pred))
print(mt.recall_score(actual, pred))
print(round(mt.f1_score(actual, pred), 2))
```

Output

```
0.75
0.6
0.67
```

Precision

Of the cases called positive, 3 of 4 were right. Use it when false alarms are costly.

Recall and F1

Of the real positives, 3 of 5 were found. F1 blends precision and recall.

~5 min

A Real Data Set

Breast cancer: benign or malignant.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, random_state=0)
clf2 = LogisticRegression(max_iter=5000)
clf2.fit(X_tr, y_tr)
print(round(clf2.score(X_te, y_te), 3))
```

Output

```
0.951
```

Test accuracy

The model labels 95.1 percent of unseen rows correctly.

max_iter

Gives the solver more steps to finish. Scaling the columns also helps it converge.

~4 min

More Than Two Classes

Logistic regression handles three or more.

```
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
m = LogisticRegression(max_iter=1000).fit(X, y)
print(m.predict(X[[0, 50, 100]]).tolist())
print(m.predict_proba(X[[0]]).round(2).tolist())
print(round(m.score(X, y), 2))
```

Output

```
[0, 1, 2]
[[0.98, 0.02, 0.0]]
0.97
```

One probability per class

The three probabilities add up to 1. The largest one wins.

Same code

Nothing changes in fit or predict. Here the first flower is class 0 with 0.98.

~4 min

Regularization With C

Smaller C means a gentler model.

```
for c in (0.01, 1, 100):
    mm = LogisticRegression(C=c)
    mm.fit(hours, passed)
    print(c, round(mm.coef_[0][0], 2))
```

Output

```
0.01 0.04
1 1.12
100 5.65
```

C

C is the inverse of regularization strength. Small C pulls the weights toward 0.

Tune it

Large C fits the training data tightly. Pick C with cross-validation.

~3 min

Logistic Regression Cheat Sheet

Match the task to the tool.

Task	Tool
Fit a classifier	`LogisticRegression().fit(X, y)`
Class and probability	`predict, predict_proba`
Change the cutoff	`p >= threshold`
Count right and wrong	`confusion_matrix`
Quality scores	`accuracy, precision, recall, f1`
Control complexity	`C, max_iter`

Check precision and recall, not only accuracy, when one class is rare.

PYTHON FOR DATA · 40 MIN

k-Nearest Neighbours

Supervised learning by similarity · Unit 4

Distance
How close are two points
`np.linalg.norm`

Voting
Neighbours decide
`n_neighbors`

Scaling
Keep features fair
`StandardScaler`

Choosing k
Test with folds
`cross_val_score`

~3 min

Four Steps of k-NN

Classify a point by looking at its neighbours.

1

Store

Keep all the training points.

`fit`

2

Measure

Find the distance to every point.

`distance`

3

Pick k

Take the k closest neighbours.

`n_neighbors`

4

Vote

The most common class wins.

`predict`

k-NN has no real training step. It remembers the data and compares at prediction time.

~3 min

Measuring Distance

How far apart are two points?

```
import numpy as np
a = np.array([1, 2])
b = np.array([4, 6])
print(np.sqrt(((a - b) ** 2).sum()))
print(np.abs(a - b).sum())
```

Output

```
5.0
7
```

Euclidean

The straight-line distance. Here 3 across and 4 up gives 5.

Manhattan

Adds the steps along each axis: 3 + 4 = 7. Like walking city blocks.

~5 min

k-NN by Hand

Sort by distance, then vote.

```
from collections import Counter
points = [([1, 1], "A"), ([2, 1], "A"),
          ([8, 9], "B"), ([9, 8], "B")]
query = np.array([2, 2])
dist = sorted((np.linalg.norm(np.array(p) - query),
lab)
              for p, lab in points)
print([f"{d:.1f} {lab}" for d, lab in dist[:3]])
votes = [lab for _, lab in dist[:3]]
print(Counter(votes).most_common(1)[0][0])
```

Output

```
['1.0 A', '1.4 A', '9.2 B']
A
```

Three nearest

With k = 3 the neighbours are A, A and B, closest first.

Majority vote

Two votes for A beat one for B, so the query is class A.

~4 min

KNeighborsClassifier

scikit-learn does the same in a few lines.

```
from sklearn.neighbors import KNeighborsClassifier
X = [[1, 1], [2, 1], [8, 9], [9, 8]]
y = ["A", "A", "B", "B"]
knn = KNeighborsClassifier(n_neighbors=3).fit(X, y)
print(knn.predict([[2, 2], [8, 8]]).tolist())
print(knn.predict_proba([[2, 2]]).round(2).tolist())
```

Output

```
['A', 'B']
[[0.67, 0.33]]
```

predict

Returns the winning class for each new point.

predict_proba

The share of neighbours in each class. Two of three are A, so 0.67.

~3 min

Look at the Neighbours

kneighbors shows who voted.

```
dist, idx = knn.kneighbors([[2, 2]])
print(dist.round(1).tolist())
print(idx.tolist())
```

Output

```
[[1.0, 1.4, 9.2]]
[[1, 0, 2]]
```

Distances

The three nearest points are 1.0, 1.4 and 9.2 away.

Indices

Rows 1, 0 and 2 of X. Use them to inspect exactly why a point got its label.

~4 min

Choosing k

Small k reacts to noise.

```
X = [[1], [2], [3], [4], [5], [6], [7], [8]]
y = [0, 0, 0, 1, 0, 1, 1, 1]
for k in (1, 3, 5):
    m = KNeighborsClassifier(n_neighbors=k).fit(X,
y)
    print(k, m.predict([[4.2]]).tolist())
```

Output

```
1 [1]
3 [0]
5 [0]
```

k = 1

Copies the single nearest point, even when it is a noisy one.

Larger k

Smooths out noise. Too large a k blurs real boundaries, so tune it.

~5 min

Scaling Matters

Big numbers dominate the distance.

```
from sklearn.preprocessing import StandardScaler
X = [[50000, 25], [52000, 60], [90000, 26]]
y = ["young", "old", "young"]
q = [[50500, 58]]
raw = KNeighborsClassifier(n_neighbors=1).fit(X, y)
print(raw.predict(q).tolist())
sc = StandardScaler().fit(X)
fixed = KNeighborsClassifier(n_neighbors=1)
fixed.fit(sc.transform(X), y)
print(fixed.predict(sc.transform(q)).tolist())
```

Output

```
['young']
['old']
```

Unscaled

Income runs in thousands, so age barely counts. A 58 year old is called young.

Scaled

Both columns count equally. Now the closest person is the other 60 year old.

~4 min

Test on Real Data

Iris with a train-test split.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_tr, y_tr)
print(round(knn.score(X_te, y_te), 3))
```

Output

```
0.978
```

Accuracy

45 test flowers, and 44 are labelled correctly.

Always split

Judge k-NN on rows it never stored, or it will simply find itself.

~5 min

Find the Best k

Try several values with cross-validation.

```
from sklearn.model_selection import cross_val_score
for k in (1, 3, 5, 9, 15, 25):
    m = KNeighborsClassifier(n_neighbors=k)
    s = cross_val_score(m, X, y, cv=5).mean()
    print(k, round(s, 3))
```

Output

```
1 0.96
3 0.967
5 0.973
9 0.973
15 0.967
25 0.96
```

Pattern

Scores rise, peak at 5 to 9, then fall again as k gets too big.

Pick the peak

Choose k from the best mean score. An odd k avoids ties in two-class problems.

~4 min

k-NN for Numbers

KNeighborsRegressor averages neighbours.

```
from sklearn.neighbors import KNeighborsRegressor
X = [[1], [2], [3], [4], [5]]
y = [10, 20, 30, 40, 50]
reg = KNeighborsRegressor(n_neighbors=2).fit(X, y)
print(reg.predict([[2.4], [10]]).tolist())
```

Output

```
[25.0, 45.0]
```

Average of neighbours

For 2.4 the neighbours are 2 and 3, so the answer is (20 + 30) / 2 = 25.

No extrapolation

For 10 it can only average the last two points, giving 45, not 100.

~3 min

Strengths and Limits

When to use k-NN.

Strength	Limit
Simple to understand	Slow when data is large
No training step	Needs scaled features
Works for classes and numbers	Hurt by noise and irrelevant columns
Handles curved boundaries	Weak with very many columns

Good for small and medium data sets. Always scale first, then tune k.

~3 min

k-NN Cheat Sheet

Match the task to the tool.

Task	Tool
Classify by neighbours	`KNeighborsClassifier(n_neighbors=k)`
Predict a number	`KNeighborsRegressor`
See the neighbours	`kneighbors`
Class shares	`predict_proba`
Keep features fair	`StandardScaler`
Choose k	`cross_val_score`

Scale the features, split the data, then try several values of k.

PYTHON FOR DATA · 40 MIN

Decision Trees

Supervised learning by asking questions · Unit 6

Gini
How mixed a group is

`gini`

Splits
Best question per node

`DecisionTreeClassifier`

Depth
Keep the tree small

`max_depth`

Importance
Which columns matter

`feature_importances_`

~3 min

Four Steps of a Tree

Predict by following a path of questions.

1

Ask

Pick a feature and a cut point.

`criterion`

2

Split

Send rows left or right.

`fit`

3

Repeat

Grow until pure or stopped.

`max_depth`

4

Predict

Follow the path to a leaf.

`predict`

A tree is a stack of if-else rules that the data wrote for you.

~3 min

Gini Impurity

How mixed is a group of labels?

```
import numpy as np
def gini(y):
    p = np.bincount(y) / len(y)
    return 1 - (p ** 2).sum()
print(gini([0, 0, 0, 0]))
print(gini([0, 0, 1, 1]))
print(round(gini([0, 0, 0, 1]), 3))
```

Output

```
0.0
0.5
0.375
```

Pure node

0.0 means every row has the same class.

Mixed node

0.5 is the worst case for two classes.

~4 min

Pick the Best Split

Lower weighted gini wins.

```
x = np.array([1, 2, 3, 4, 5, 6])
y = np.array([0, 0, 0, 1, 1, 1])
for t in (2.5, 3.5):
    L, R = y[x < t], y[x >= t]
    w = (len(L)*gini(L) + len(R)*gini(R)) / 6
    print(t, round(w, 3))
```

Output

```
2.5 0.25
3.5 0.0
```

Weighted score

Each side counts in proportion to its size.

Cut at 3.5

Both sides are pure, so the score is 0.0.

~3 min

DecisionTreeClassifier

scikit-learn grows the tree for you.

```
from sklearn.tree import DecisionTreeClassifier
X = [[1], [2], [3], [4], [5], [6]]
y = [0, 0, 0, 1, 1, 1]
tree = DecisionTreeClassifier().fit(X, y)
print(tree.predict([[2.5], [4.5]]).tolist())
print(tree.get_depth(), tree.get_n_leaves())
```

Output

```
[0, 1]
1 2
```

predict

2.5 falls left of the cut and 4.5 falls right.

Depth and leaves

One question and two leaf nodes is all it needs.

~3 min

Read the Rules

export_text prints the tree as text.

```
from sklearn.tree import export_text
print(export_text(tree, feature_names=["x"]))
```

Output

```
|--- x <= 3.50
|   |--- class: 0
|--- x >  3.50
|   |--- class: 1
```

Each line is a test

Rows that pass go down. Others take the next branch.

Leaves give classes

The last line of each path is the prediction.

~4 min

Test on Real Data

Breast cancer with a train-test split.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0)
t = DecisionTreeClassifier(random_state=0)
t.fit(X_tr, y_tr)
tr, te = t.score(X_tr, y_tr), t.score(X_te, y_te)
print(round(tr, 3), round(te, 3))
```

Output

```
1.0 0.912
```

Train 1.0

The tree memorises every training row.

Test 0.912

Fewer new rows are right. That gap is overfitting.

~4 min

Limit the Depth

A shorter tree generalises better.

```
for d in (None, 3, 1):
    m = DecisionTreeClassifier(
        max_depth=d, random_state=0).fit(X_tr, y_tr)
    tr = round(m.score(X_tr, y_tr), 3)
    te = round(m.score(X_te, y_te), 3)
    print(d, tr, te)
```

Output

```
None 1.0 0.912
3 0.967 0.947
1 0.93 0.895
```

Deep tree

None grows until every leaf is pure. Train is 1.0, test only 0.912.

Depth 3

Train drops a little, test rises to 0.947. Depth 1 is too simple.

~4 min

Find the Best Depth

Try several values with cross-validation.

```
from sklearn.model_selection import cross_val_score
for d in (1, 2, 3, 6):
    m = DecisionTreeClassifier(
        max_depth=d, random_state=0)
    s = cross_val_score(m, X, y, cv=5).mean()
    print(d, round(s, 3))
```

Output

```
1 0.9
2 0.928
3 0.917
6 0.914
```

Pattern

Scores peak at depth 2, then slip and stay flat.

Pick the peak

Choose depth from the best mean score, not from one split.

~3 min

Feature Importance

Which columns drive the splits?

```
from sklearn.datasets import load_iris
iris = load_iris()
m = DecisionTreeClassifier(max_depth=3,
random_state=0)
m.fit(iris.data, iris.target)
print(m.feature_importances_.round(2).tolist())
```

Output

```
[0.0, 0.0, 0.05, 0.95]
```

Order

Sepal length, sepal width, petal length, petal width.

Petal width

It scores 0.95, so one column does almost all the work.

~3 min

Trees for Numbers

DecisionTreeRegressor averages a leaf.

```
from sklearn.tree import DecisionTreeRegressor
X = [[1], [2], [3], [4], [5], [6]]
y = [10, 12, 11, 30, 32, 31]
reg = DecisionTreeRegressor(max_depth=1).fit(X, y)
print(reg.predict([[2.5], [5.5], [100]]).tolist())
```

Output

```
[11.0, 31.0, 31.0]
```

Leaf average

Rows 10, 12, 11 in one leaf give 11. The other gives 31.

No extrapolation

For 100 it repeats the last leaf, 31, and never goes higher.

~3 min

Strengths and Limits

When to use a decision tree.

Strength	Limit
Easy to read and explain	Overfits when grown deep
No scaling needed	Small data changes can change the tree
Handles classes and numbers	Cannot extrapolate beyond the data
Captures non-linear patterns	Single trees are less accurate than ensembles

Great for explainable models. Limit the depth, or move to a forest.

~3 min

Decision Tree Cheat Sheet

Match the task to the tool.

Task	Tool
Classify with rules	`DecisionTreeClassifier(max_depth=d)`
Predict a number	`DecisionTreeRegressor`
Print the rules	`export_text`
Which columns matter	`feature_importances_`
Choose the depth	`cross_val_score`

Split the data, limit the depth, then check the test score.

PYTHON FOR DATA · 40 MIN

k-Means Clustering

Unsupervised learning by grouping · Unit 7

Centroids
The centre of each group

`cluster_centers_`

Assign
Nearest centre wins

`labels_`

Inertia
How tight the groups are

`inertia_`

Choosing k
Elbow and silhouette

`silhouette_score`

~3 min

Four Steps of k-Means

Group points without any labels.

1

Start

Pick k starting centres.

`n_clusters`

2

Assign

Each point joins its nearest centre.

`labels_`

3

Update

Move each centre to its group mean.

`cluster_centers_`

4

Repeat

Stop when nothing moves.

`n_iter_`

k-means has no answer key. It finds groups from distances alone.

~3 min

Distance to Centres

Measure every point against every centre.

```
import numpy as np
X = np.array([[1, 1], [2, 1], [8, 9], [9, 8]])
C = np.array([[1, 1], [9, 9]])
d = np.linalg.norm(X[:, None] - C, axis=2)
print(d.round(1).tolist())
print(d.argmin(axis=1).tolist())
```

Output

```
[[0.0, 11.3], [1.0, 10.6], [10.6, 1.0], [10.6, 1.0]]
[0, 0, 1, 1]
```

Rows are points

Each row holds the distance to centre 0, then centre 1.

argmin

The smaller distance wins, so the points go 0, 0, 1, 1.

~4 min

Move the Centres

Each centre becomes its group's mean.

```
lab = d.argmin(axis=1)
new = [X[lab == k].mean(axis=0) for k in (0, 1)]
print(np.array(new).tolist())
```

Output

```
[[1.5, 1.0], [8.5, 8.5]]
```

Group means

Points 1 and 2 average to (1.5, 1.0).

Repeat

Assign again with the new centres until nothing changes.

~3 min

KMeans

scikit-learn does the loop for you.

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=0)
km.fit(X)
print(km.labels_.tolist())
print(km.predict([[2, 2], [8, 8]]).tolist())
```

Output

```
[1, 1, 0, 0]
[1, 0]
```

labels_

One cluster number per row. The numbers are arbitrary.

predict

Sends new points to the nearest centre.

~3 min

Centres and Inertia

What the fitted model keeps.

```
print(km.cluster_centers_.tolist())
print(km.inertia_)
gap = X - km.cluster_centers_[km.labels_]
print((gap ** 2).sum())
```

Output

```
[[8.5, 8.5], [1.5, 1.0]]
1.5
1.5
```

cluster_centers_

One row per cluster, in the same columns as X.

inertia_

The sum of squared gaps to the centre. Lower is tighter.

~4 min

Cluster Real Blobs

Three groups hidden in 150 points.

```
from sklearn.datasets import make_blobs
Xb, _ = make_blobs(n_samples=150, centers=3,
                   random_state=0)
km = KMeans(n_clusters=3, n_init=10, random_state=0)
km.fit(Xb)
print(np.bincount(km.labels_).tolist())
print(round(km.inertia_, 1))
```

Output

```
[48, 50, 52]
262.7
```

Sizes

The groups hold 48, 50 and 52 points.

Inertia

262.7 is the total squared distance to the centres.

~4 min

Scaling Matters

Big numbers dominate the distance.

```
from sklearn.preprocessing import StandardScaler
P = np.array([[50000, 25], [52000, 60],
              [90000, 26], [88000, 58]])
raw = KMeans(2, n_init=10, random_state=0).fit(P)
print(raw.labels_.tolist())
sc = StandardScaler().fit_transform(P)
fixed = KMeans(2, n_init=10, random_state=0).fit(sc)
print(fixed.labels_.tolist())
```

Output

```
[1, 1, 0, 0]
[0, 1, 0, 1]
```

Unscaled

Income runs in thousands, so the split is by income only.

Scaled

Both columns count equally. Now the split is young and old.

~4 min

The Elbow

Inertia always falls as k grows.

```
for k in range(1, 7):
    m = KMeans(k, n_init=10, random_state=0)
    m.fit(Xb)
    print(k, round(m.inertia_))
```

Output

```
1 942
2 476
3 263
4 212
5 174
6 151
```

Pattern

The drop is big up to k = 3, then it flattens out.

Pick the bend

Choose k at the elbow, where extra groups stop paying off.

~3 min

Silhouette Score

How well each point fits its own group.

```
from sklearn.metrics import silhouette_score
for k in (2, 3, 4, 5):
    m = KMeans(k, n_init=10, random_state=0)
    lab = m.fit_predict(Xb)
    print(k, round(silhouette_score(Xb, lab), 3))
```

Output

```
2 0.443
3 0.477
4 0.419
5 0.413
```

Higher is better

The score runs from -1 to 1. Near 1 means clean groups.

Peak at 3

It agrees with the elbow, so k = 3.

~3 min

Assign New Points

Reuse the fitted centres.

```
km = KMeans(n_clusters=2, n_init=10, random_state=0)
km.fit(X)
q = [[2, 2], [7, 7]]
print(km.predict(q).tolist())
print(km.transform(q).round(1).tolist())
```

Output

```
[1, 0]
[[9.2, 1.1], [2.1, 8.1]]
```

predict

[2, 2] joins cluster 1 and [7, 7] joins cluster 0.

transform

Gives the distance from each point to every centre.

~3 min

Strengths and Limits

When to use k-means.

Strength	Limit
Simple and fast	You must choose k yourself
Scales to large data sets	Needs scaled features
Easy to explain	Finds round groups of similar size
Centres are easy to read	Start points can change the result

Scale first, try several k, and keep n_init above 1 for stable results.

~3 min

k-Means Cheat Sheet

Match the task to the tool.

Task	Tool
Group the rows	`KMeans(n_clusters=k, n_init=10)`
Get the groups	`labels_`
See the centres	`cluster_centers_`
Judge the tightness	`inertia_`
Choose k	`silhouette_score`

Scale the features, try several values of k, then check the silhouette.

PYTHON FOR DATA · 40 MIN

Hierarchical Clustering

Unsupervised learning by merging · Unit 8

Linkage
How groups are compared
`linkage`

Dendrogram
The tree of merges
`dendrogram`

Cutting
Choose the groups
`fcluster`

Agglomerative
The scikit-learn way
`AgglomerativeClustering`

~3 min

Four Steps of Merging

Build a tree by joining the closest groups.

1

Start

Every point is its own cluster.

`n clusters`

2

Merge

Join the two closest clusters.

`linkage`

3

Repeat

Until one cluster is left.

`n - 1 merges`

4

Cut

Slice the tree at a height.

`fcluster`

Unlike k-means, you do not pick k first. You build the whole tree, then cut.

~3 min

Pairwise Distances

Every merge starts from distances.

```
import numpy as np
from scipy.spatial.distance import pdist
X = np.array([[1, 1], [2, 1], [8, 9], [9, 8]])
print(pdist(X).round(1).tolist())
```

Output

```
[1.0, 10.6, 10.6, 10.0, 9.9, 1.4]
```

pdist

Gives the distance for each pair of 4 points: 6 values.

Reading it

Pairs (0,1) and (2,3) are close. The rest are far apart.

~4 min

The Linkage Matrix

Each row is one merge.

```
from scipy.cluster.hierarchy import linkage
Z = linkage(X, method="average")
for row in Z.round(1):
    print(row.tolist())
```

Output

```
[0.0, 1.0, 1.0, 2.0]
[2.0, 3.0, 1.4, 2.0]
[4.0, 5.0, 10.3, 4.0]
```

Four columns

The two clusters joined, their distance, and the new size.

Row 3

Clusters 4 and 5 join at 10.3 to make a group of 4.

~3 min

Cut the Tree

Choose groups by count or by height.

```
from scipy.cluster.hierarchy import fcluster
two = fcluster(Z, t=2, criterion="maxclust")
print(two.tolist())
near = fcluster(Z, t=1.2, criterion="distance")
print(near.tolist())
```

Output

```
[1, 1, 2, 2]
[1, 1, 2, 3]
```

maxclust

t = 2 asks for exactly two groups.

distance

t = 1.2 cuts at that height. Point 3 stays alone.

~3 min

Read the Dendrogram

The tree behind the merges.

```
from scipy.cluster.hierarchy import dendrogram
d = dendrogram(Z, no_plot=True)
print(d["leaves"])
print(Z[:, 2].round(1).tolist())
```

Output

```
[0, 1, 2, 3]
[1.0, 1.4, 10.3]
```

leaves

The order of points along the bottom of the plot.

Heights

Low merges join close points. A tall merge joins far groups.

~4 min

Linkage Methods

How far apart are two groups?

```
P = np.array([[0], [1], [2], [3], [7], [8], [9]])
for m in ("single", "complete", "average", "ward"):
    Zm = linkage(P, method=m)
    print(m, round(Zm[-1, 2], 1))
```

Output

```
single 4.0
complete 9.0
average 6.5
ward 12.0
```

single and complete

Single uses the closest pair. Complete uses the farthest.

average and ward

Average takes the mean gap. Ward limits the added spread.

~3 min

AgglomerativeClustering

scikit-learn builds and cuts in one step.

```
from sklearn.cluster import AgglomerativeClustering
agg = AgglomerativeClustering(n_clusters=2)
agg.fit(X)
print(agg.labels_.tolist())
print(agg.n_leaves_, agg.n_clusters_)
```

Output

```
[1, 1, 0, 0]
4 2
```

n_clusters

Where to cut the tree. The default linkage is ward.

Attributes

n_leaves_ counts the points. labels_ gives each cluster.

~3 min

Cluster Real Blobs

Three groups hidden in 90 points.

```
from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score
Xb, yb = make_blobs(n_samples=90, centers=3,
                    cluster_std=0.6, random_state=0)
agg = AgglomerativeClustering(n_clusters=3)
lab = agg.fit_predict(Xb)
print(np.bincount(lab).tolist())
print(round(adjusted_rand_score(yb, lab), 2))
```

Output

```
[30, 31, 29]
0.97
```

Sizes

The groups hold 30, 31 and 29 points.

Adjusted Rand

0.97 means the groups match the true ones closely.

~4 min

Scaling Matters

Big numbers dominate the distance.

```
from sklearn.preprocessing import StandardScaler
P = np.array([[50000, 25], [52000, 60],
              [90000, 26], [88000, 58]])
raw = AgglomerativeClustering(2).fit(P)
print(raw.labels_.tolist())
sc = StandardScaler().fit_transform(P)
fixed = AgglomerativeClustering(2).fit(sc)
print(fixed.labels_.tolist())
```

Output

```
[0, 0, 1, 1]
[0, 1, 0, 1]
```

Unscaled

Income runs in thousands, so the split is by income only.

Scaled

Both columns count equally. Now the split is young and old.

~4 min

Find the Gap

Where should we cut?

```
Zb = linkage(Xb, method="ward")
heights = Zb[-4:, 2].round(1)
print(heights.tolist())
print(np.diff(heights).round(1).tolist())
```

Output

```
[4.1, 4.2, 15.7, 22.0]
[0.1, 11.5, 6.3]
```

Heights

The last four merges happen at 4.1, 4.2, 15.7 and 22.0.

Biggest jump

Cutting below 15.7 leaves 3 groups, so k = 3.

~3 min

Strengths and Limits

When to use hierarchical clustering.

Strength	Limit
No need to pick k first	Slow and memory heavy on big data
The dendrogram shows structure	A merge can never be undone
Works with several linkages	Needs scaled features
Same result on every run	Sensitive to noise and outliers

Great for small data sets. Scale first, then choose a linkage and cut the tree.

~3 min

Hierarchical Cheat Sheet

Match the task to the tool.

Task	Tool
Build the merge tree	`linkage(X, method="ward")`
Cut into groups	`fcluster(Z, t, criterion)`
Draw the tree	`dendrogram(Z)`
Cluster in one step	`AgglomerativeClustering(n_clusters=k)`
Compare linkages	`single, complete, average, ward`

Scale the features, build the tree, then cut where the merge heights jump.

PYTHON FOR DATA · 40 MIN

Dimensionality Reduction with PCA

Unsupervised learning by compressing · Unit 9

Components
New axes of variation
`components_`

Variance
How much each one keeps
`explained_variance_ratio_`

Projection
Fewer columns, same rows
`transform`

Scaling
Keep features fair
`StandardScaler`

~3 min

Four Steps of PCA

Keep the directions where the data spreads most.

1

Centre

Subtract each column's mean.

`mean_`

2

Find axes

Directions of greatest spread.

`components_`

3

Rank

Order them by variance kept.

`explained_variance_`

4

Project

Keep the top few components.

`transform`

PCA turns many correlated columns into a few new ones that keep most of the information.

~3 min

Covariance by Hand

Spread and overlap between columns.

```
import numpy as np
X = np.array([[1, 2], [2, 4], [3, 5], [4, 7]])
Xc = X - X.mean(axis=0)
cov = np.cov(Xc.T)
print(cov.round(2).tolist())
vals, vecs = np.linalg.eigh(cov)
print(vals.round(2).tolist())
```

Output

```
[[1.67, 2.67], [2.67, 4.33]]
[0.02, 5.98]
```

Covariance

The large off-diagonal 2.67 shows the columns rise together.

Eigenvalues

5.98 is the big spread. 0.02 is almost nothing.

~4 min

The Principal Direction

Project the points onto the top axis.

```
pc1 = vecs[:, -1]
print(pc1.round(2).tolist())
print((Xc @ pc1).round(2).tolist())
print(round(vals[-1] / vals.sum(), 3))
```

Output

```
[0.53, 0.85]
[-2.92, -0.69, 0.69, 2.92]
0.997
```

pc1

The eigenvector with the largest eigenvalue. It points up and right.

One number each

Each point becomes a single value. It keeps 99.7% of the spread.

~3 min

PCA in scikit-learn

Same answer with far less code.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=1)
Z = pca.fit_transform(X)
print(Z.ravel().round(2).tolist())
print(pca.components_.round(2).tolist())
ev = pca.explained_variance_ratio_
print(ev.round(3).tolist())
```

Output

```
[-2.92, -0.69, 0.69, 2.92]
[[0.53, 0.85]]
[0.997]
```

fit_transform

Centres, finds the axis and projects in one call.

Sign can flip

The direction may point the other way. The spread is the same.

~3 min

Explained Variance

How much does each component keep?

```
from sklearn.datasets import load_iris
Xi = load_iris().data
pca = PCA().fit(Xi)
ratio = pca.explained_variance_ratio_
print(ratio.round(3).tolist())
print(np.cumsum(ratio).round(3).tolist())
```

Output

```
[0.925, 0.053, 0.017, 0.005]
[0.925, 0.978, 0.995, 1.0]
```

Ratios

PC1 alone keeps 92.5% of the variance in iris.

Cumulative

Two components keep 97.8%, so 4 columns shrink to 2.

~4 min

Read the Components

Each row is a recipe of the original columns.

```
pca2 = PCA(n_components=2).fit(Xi)
for row in pca2.components_.round(2):
    print(row.tolist())
print(pca2.transform(Xi).shape)
```

Output

```
[0.36, -0.08, 0.86, 0.36]
[0.66, 0.73, -0.17, -0.08]
(150, 2)
```

PC1

Petal length weighs most at 0.86, then sepal length and petal width.

PC2

Mostly the two sepal columns. Each point has 2 new values.

~3 min

Scaling Matters

Big numbers dominate the variance.

```
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
Xw = load_wine().data
r = PCA(2).fit(Xw).explained_variance_ratio_
print(r.round(3).tolist())
Xs = StandardScaler().fit_transform(Xw)
r = PCA(2).fit(Xs).explained_variance_ratio_
print(r.round(3).tolist())
```

Output

```
[0.998, 0.002]
[0.362, 0.192]
```

Unscaled

One wine column has huge values and takes 99.8% of PC1.

Scaled

Every column counts. PC1 keeps 36.2% and PC2 keeps 19.2%.

~3 min

How Many Components?

Ask for a share of the variance.

```
for goal in (0.60, 0.80, 0.95):
    p = PCA(n_components=goal).fit(Xs)
    print(goal, p.n_components_)
```

Output

```
0.6 3
0.8 5
0.95 10
```

Float n_components

0.95 means keep enough components for 95% of the variance.

The trade-off

13 wine columns need 10 components to keep 95%.

~4 min

Rebuild the Data

inverse_transform goes back to 4 columns.

```
Z = pca2.transform(Xi)
back = pca2.inverse_transform(Z)
print(Z.shape, back.shape)
err = ((Xi - back) ** 2).mean()
print(round(err, 3))
```

Output

```
(150, 2) (150, 4)
0.025
```

Round trip

150 rows by 2 columns grow back to 150 by 4.

Small loss

The mean squared error is 0.025. The dropped detail is lost.

~4 min

PCA in a Pipeline

Compress before the model.

```
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
yw = load_wine().target
for n in (2, 5, 13):
    pipe = make_pipeline(StandardScaler(), PCA(n),
                         KNeighborsClassifier())
    s = cross_val_score(pipe, Xw, yw, cv=5).mean()
    print(n, round(s, 3))
```

Output

```
2 0.966
5 0.961
13 0.949
```

make_pipeline

Scale, reduce and classify in one object. Safe for cross-validation.

Fewer can be better

Two components score 0.966. All 13 score 0.949.

~3 min

Strengths and Limits

When to use PCA.

Strength	Limit
Cuts many columns to a few	New columns are harder to explain
Removes correlated noise	Only finds straight-line patterns
Speeds up later models	Needs scaled features
Easy to plot in 2D	Some information is always lost

Scale first, then keep enough components for about 95% of the variance.

~3 min

PCA Cheat Sheet

Match the task to the tool.

Task	Tool
Fit and project	`PCA(n_components=k).fit_transform(X)`
See the share kept	`explained_variance_ratio_`
See the axes	`components_`
Go back to the columns	`inverse_transform`
Keep features fair	`StandardScaler`

Scale the features, fit PCA, then check how much variance you kept.

PYTHON FOR DATA · 40 MIN

Overfitting and Underfitting

Model evaluation · Unit 10

Underfit
Too simple to learn

`high bias`

Overfit
Memorises the noise

`high variance`

Validation
Score on unseen rows

`train_test_split`

Curves
Watch the gap

`validation_curve`

~3 min

Four Steps to Check the Fit

Compare the score on seen rows with unseen rows.

1

Split

Hold out some rows for testing.

`train_test_split`

2

Fit

Learn from the training rows only.

`fit`

3

Compare

Train score against test score.

`score`

4

Adjust

Simplify, constrain or add data.

`max_depth`

A good model scores well on rows it has never seen, not just on rows it has memorised.

~3 min

Underfitting by Hand

A straight line cannot follow a curve.

```
import numpy as np
x = np.arange(10)
y = x ** 2
for d in (1, 2):
    c = np.polyfit(x, y, d)
    err = np.mean((np.polyval(c, x) - y) ** 2)
    print(d, round(err, 2))
```

Output

```
1 52.8
2 0.0
```

Degree 1

A line leaves a large error of 52.8, even on the training points.

Degree 2

A curve matches y = x squared, so the error is 0.0.

~4 min

Overfitting by Hand

A wiggly curve chases the noise.

```
rng = np.random.default_rng(1)
x = np.linspace(0, 1, 10)
y = 2 * x + rng.normal(0, 0.2, 10)
xt = np.linspace(0.05, 0.95, 10)
yt = 2 * xt + rng.normal(0, 0.2, 10)
for d in (1, 9):
    c = np.polyfit(x, y, d)
    tr = np.mean((np.polyval(c, x) - y) ** 2)
    te = np.mean((np.polyval(c, xt) - yt) ** 2)
    print(d, round(tr, 3), round(te, 3))
```

Output

```
1 0.016 0.014
9 0.0 0.122
```

Degree 9

It hits every training point, so the error is 0.0.

Fresh points

Test error jumps to 0.122. The line scored 0.014.

~3 min

Train vs Test Score

Noisy moons with a train-test split.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
X, y = make_moons(300, noise=0.5, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0)
t = DecisionTreeClassifier(random_state=0)
t.fit(X_tr, y_tr)
print(round(t.score(X_tr, y_tr), 3))
print(round(t.score(X_te, y_te), 3))
```

Output

```
1.0
0.778
```

Train 1.0

The tree memorises every training row.

Test 0.778

A gap of 0.222 signals overfitting.

~4 min

Too Simple, Too Complex

Vary the depth and compare.

```
for d in (1, 3, None):
    m = DecisionTreeClassifier(
        max_depth=d, random_state=0).fit(X_tr, y_tr)
    tr = round(m.score(X_tr, y_tr), 3)
    te = round(m.score(X_te, y_te), 3)
    print(d, tr, te)
```

Output

```
1 0.8 0.744
3 0.852 0.856
None 1.0 0.778
```

Depth 1: underfit

Both scores are low: 0.8 and 0.744.

Depth 3: good fit

Test peaks at 0.856. None overfits: 1.0 against 0.778.

~4 min

The Validation Curve

Score every depth with cross-validation.

```
from sklearn.model_selection import validation_curve
ds = [1, 2, 4, 8, 16]
tr, va = validation_curve(
    DecisionTreeClassifier(random_state=0), X, y,
    param_name="max_depth", param_range=ds, cv=5)
for d, a, b in zip(ds, tr.mean(1), va.mean(1)):
    print(d, round(a, 3), round(b, 3))
```

Output

```
1 0.783 0.78
2 0.857 0.85
4 0.867 0.837
8 0.961 0.81
16 1.0 0.78
```

Pattern

Train keeps rising to 1.0. Validation peaks at depth 2, then falls.

Pick the peak

Choose the setting with the best validation score.

~4 min

The Learning Curve

Does more data close the gap?

```
from sklearn.model_selection import learning_curve
n, tr, va = learning_curve(
    DecisionTreeClassifier(random_state=0), X, y,
    train_sizes=[0.2, 0.5, 1.0], cv=5)
for k, a, b in zip(n, tr.mean(1), va.mean(1)):
    print(k, round(a, 3), round(b, 3))
```

Output

```
48 1.0 0.743
120 1.0 0.773
240 1.0 0.78
```

Train stays 1.0

A deep tree memorises any amount of data.

Validation climbs

From 0.743 to 0.78. More data helps, but slowly.

~3 min

Add a Constraint

Force leaves to hold more rows.

```
for leaf in (1, 5, 20, 80):
    m = DecisionTreeClassifier(
        min_samples_leaf=leaf, random_state=0)
    m.fit(X_tr, y_tr)
    tr = round(m.score(X_tr, y_tr), 3)
    te = round(m.score(X_te, y_te), 3)
    print(leaf, tr, te)
```

Output

```
1 1.0 0.778
5 0.876 0.8
20 0.8 0.744
80 0.8 0.744
```

Leaf of 5

Train drops to 0.876 and test rises to 0.8.

Too strict

At 20 or more both scores fall. That is underfitting again.

~3 min

Cross-Validation

A steadier score than one split.

```
from sklearn.model_selection import cross_val_score
for d in (3, None):
    m = DecisionTreeClassifier(
        max_depth=d, random_state=0)
    s = cross_val_score(m, X, y, cv=5)
    print(d, round(s.mean(), 3), round(s.std(), 3))
```

Output

```
3 0.85 0.056
None 0.78 0.036
```

Mean and spread

Five folds give an average and a standard deviation.

Depth 3 wins

0.85 beats the unlimited tree's 0.78.

~3 min

Search the Settings

GridSearchCV tries them all.

```
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(
    DecisionTreeClassifier(random_state=0),
    {"max_depth": [1, 2, 3, 4, 6, 8]}, cv=5)
grid.fit(X_tr, y_tr)
print(grid.best_params_)
print(round(grid.best_score_, 3))
print(round(grid.score(X_te, y_te), 3))
```

Output

```
{'max_depth': 2}
0.829
0.856
```

best_params_

Depth 2 scores best across 5 folds at 0.829.

Test once

Use the held-out rows only after tuning: 0.856.

~3 min

Underfit vs Overfit

How to tell them apart.

Sign	Underfitting	Overfitting
Train score	Low	Very high
Test score	Low, close to train	Much lower than train
Cause	Model too simple	Too complex or too little data
Fix	Add features or depth	Simplify, constrain or add data

Watch the gap between train and test. Tune until both are high and close.

~3 min

Evaluation Cheat Sheet

Match the task to the tool.

Task	Tool
Hold out test rows	`train_test_split`
Get a steady score	`cross_val_score`
Tune one setting	`validation_curve`
Check if more data helps	`learning_curve`
Search several settings	`GridSearchCV`

Split first, compare train and test, then tune with cross-validation.

PYTHON FOR DATA · 40 MIN

Confusion Matrix

Counting right and wrong predictions · Unit 4

Four outcomes
TP, TN, FP, FN
`confusion_matrix`

Core scores
Accuracy and precision
`accuracy_score`

Recall and F1
Catch what matters
`recall_score`

Many classes
Three or more labels
`normalize`

~3 min

Four Possible Outcomes

Every prediction lands in one of four boxes.

TP

True Positive

Predicted yes, and it was yes.

`a hit`

TN

True Negative

Predicted no, and it was no.

`a correct reject`

FP

False Positive

Predicted yes, but it was no.

`a false alarm`

FN

False Negative

Predicted no, but it was yes.

`a miss`

True means the model was right. False means it was wrong. Positive or negative is what it predicted.

~4 min

Reading the Matrix

Rows are the real class. Columns are the prediction.

	Predicted 0	Predicted 1
Real 0	3 TN	1 FP
Real 1	2 FN	4 TP

The diagonal

TN and TP sit on the diagonal. These are the correct calls: 3 + 4 = 7 of 10.

Off the diagonal

FP and FN are the mistakes. A false alarm costs differently from a miss.

~4 min

Counting by Hand

Pair each real label with its prediction.

```
actual = [1, 1, 1, 1, 1, 1, 0, 0, 0, 0]
pred   = [1, 1, 1, 1, 0, 0, 0, 0, 0, 1]
pairs = list(zip(actual, pred))
tp = pairs.count((1, 1))
tn = pairs.count((0, 0))
fp = pairs.count((0, 1))
fn = pairs.count((1, 0))
print(tp, tn, fp, fn)
```

Output

```
4 3 1 2
```

Pairs

zip joins each real label with its prediction. (0, 1) means real 0, predicted 1.

The four counts

4 hits, 3 correct rejects, 1 false alarm and 2 misses. They add up to 10.

~4 min

confusion_matrix

scikit-learn counts them for you.

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(actual, pred)
print(cm.tolist())
tn, fp, fn, tp = cm.ravel()
print(tn, fp, fn, tp)
```

Output

```
[[3, 1], [2, 4]]
3 1 2 4
```

Layout

Row 0 is real 0 and row 1 is real 1. Columns are the predictions in the same order.

ravel

Flattens the matrix in the order TN, FP, FN, TP. A handy way to name the four counts.

~3 min

Accuracy

The share of correct predictions.

```
from sklearn import metrics as mt
print((tp + tn) / cm.sum())
print(mt.accuracy_score(actual, pred))
```

Output

```
0.7
0.7
```

Formula

(TP + TN) divided by all predictions. Here (4 + 3) / 10 = 0.7.

Watch out

With rare positives, a model that always says no can still score high accuracy.

~3 min

Precision

When it says yes, how often is it right?

```
print(tp / (tp + fp))
print(mt.precision_score(actual, pred))
```

Output

```
0.8
0.8
```

Formula

TP divided by TP + FP. Of 5 predicted positives, 4 were real: 0.8.

Use it when

False alarms are costly, such as flagging good email as spam.

~3 min

Recall

Of the real yeses, how many were found?

```
print(round(tp / (tp + fn), 2))
print(round(mt.recall_score(actual, pred), 2))
```

Output

```
0.67
0.67
```

Formula

TP divided by TP + FN. Of 6 real positives, 4 were found: 0.67.

Use it when

Misses are costly, such as screening for a disease.

~4 min

F1 and Specificity

One blended score and the negative side.

```
p = tp / (tp + fp)
r = tp / (tp + fn)
print(round(2 * p * r / (p + r), 2))
print(round(mt.f1_score(actual, pred), 2))
print(tn / (tn + fp))
```

Output

```
0.73
0.73
0.75
```

F1

The harmonic mean of precision and recall. It is low if either one is low.

Specificity

TN divided by TN + FP: how many real negatives were cleared. Here 3 of 4.

~4 min

The Report in Code

classification_report gathers the scores.

```
rep = mt.classification_report(
    actual, pred, output_dict=True)
for key in ("precision", "recall", "f1-score"):
    print(key, round(rep["1"][key], 2))
print(rep["1"]["support"])
```

Output

```
precision 0.8
recall 0.67
f1-score 0.73
6.0
```

Per class

rep["1"] holds the scores for class 1. rep["0"] holds class 0.

support

The number of real rows in the class. Class 1 has 6.

~5 min

Changing the Threshold

A probability becomes a yes at a cutoff.

```
import numpy as np
probs = np.array([0.9, 0.8, 0.7, 0.6, 0.45,
                  0.35, 0.4, 0.2, 0.1, 0.55])
for t in (0.5, 0.3):
    guess = (probs >= t).astype(int)
    cm_t = confusion_matrix(actual, guess)
    print(t, cm_t.tolist())
```

Output

```
0.5 [[3, 1], [2, 4]]
0.3 [[2, 2], [0, 6]]
```

Lower cutoff

At 0.3 there are no misses (FN = 0), but false alarms double from 1 to 2.

Trade-off

Raising recall usually lowers precision. Pick the cutoff that suits the cost of each error.

~4 min

Three Classes

The matrix grows to 3 by 3.

```
actual3 = [0, 0, 0, 1, 1, 1, 2, 2, 2, 2]
pred3 = [0, 0, 1, 1, 1, 2, 2, 2, 2, 0]
cm3 = confusion_matrix(actual3, pred3)
print(cm3)
print(cm3.diagonal().sum() / cm3.sum())
rec = cm3.diagonal() / cm3.sum(axis=1)
print(rec.round(2).tolist())
```

Output

```
[[2 1 0]
 [0 2 1]
 [1 0 3]]
0.7
[0.67, 0.67, 0.75]
```

Diagonal

The diagonal holds the correct calls: 2 + 2 + 3 = 7 of 10, so accuracy is 0.7.

Recall per class

Each diagonal value over its row total: 2/3, 2/3 and 3/4.

~4 min

Labels and Percentages

Make the matrix easy to read.

```
import pandas as pd
df = pd.DataFrame(cm, index=["real 0", "real 1"],
                  columns=["pred 0", "pred 1"])
print(df)
cmn = confusion_matrix(actual, pred,
                       normalize="true")
print(cmn.round(2).tolist())
```

Output

```
        pred 0  pred 1
real 0       3       1
real 1       2       4
[[0.75, 0.25], [0.33, 0.67]]
```

DataFrame

Labelled rows and columns make the counts easy to read.

normalize

normalize="true" turns each row into shares. Real 1 rows: 0.33 missed, 0.67 found.

~3 min

Which Metric Fits?

Pick the score that matches the cost.

Situation	Metric
Classes are balanced	Accuracy
False alarms are costly	Precision
Misses are costly	Recall
Both errors matter	F1
You want to clear negatives	Specificity
Cutoff is up to you	Matrix at each threshold

Accuracy alone can hide a model that never finds the rare class.

~3 min

Confusion Matrix Cheat Sheet

Match the task to the tool.

Task	Tool
Count the four outcomes	`confusion_matrix(actual, pred)`
Name the counts	`cm.ravel()`
Share correct	`accuracy_score`
Positive calls that are right	`precision_score`
Real positives found	`recall_score, f1_score`
Full per-class scores	`classification_report`

Read the matrix first, then pick the metric that fits your problem.

PYTHON FOR DATA · 40 MIN

Classification Metrics and ROC

Accuracy, precision, recall, F1-score and the ROC curve · Unit 4

Accuracy
Share correct
`accuracy_score`

Precision
Trust a yes
`precision_score`

Recall and F1
Find the positives
`f1_score`

ROC curve
All cutoffs at once
`roc_auc_score`

~3 min

Four Scores and a Curve

Each one answers a different question.

A

Accuracy

How many of all calls are right?

`(TP+TN) / all`

P

Precision

Of predicted yes, how many are real?

`TP / (TP+FP)`

R

Recall

Of real yes, how many were found?

`TP / (TP+FN)`

F1

F1-Score

One balance of precision and recall.

`2PR / (P+R)`

The ROC curve then shows how these trade off across every possible cutoff.

~4 min

Accuracy

The share of all predictions that are right.

```
from sklearn import metrics as mt
actual = [1, 1, 1, 1, 1, 1, 0, 0, 0, 0]
pred   = [1, 1, 1, 1, 0, 0, 0, 0, 0, 1]
cm = mt.confusion_matrix(actual, pred)
tn, fp, fn, tp = cm.ravel()
print(tp, tn, fp, fn)
print((tp + tn) / cm.sum())
print(mt.accuracy_score(actual, pred))
```

Output

```
4 3 1 2
0.7
0.7
```

Formula

Correct calls (TP + TN) divided by all calls. Here (4 + 3) / 10 = 0.7.

Counts first

ravel names the four counts. Every score on the next slides uses them.

~4 min

The Accuracy Trap

High accuracy can hide a useless model.

```
a2 = [0] * 95 + [1] * 5
p2 = [0] * 100
print(mt.accuracy_score(a2, p2))
print(mt.recall_score(a2, p2))
print(mt.f1_score(a2, p2, zero_division=0))
```

Output

```
0.95
0.0
0.0
```

Rare positives

Only 5 of 100 cases are positive. Always answering no is right 95 times.

Look deeper

Recall and F1 are 0.0, so this model finds none of the cases that matter.

~3 min

Precision

When the model says yes, can you trust it?

```
print(tp + fp)
print(tp / (tp + fp))
print(mt.precision_score(actual, pred))
```

Output

```
5
0.8
0.8
```

Formula

TP divided by TP + FP. The model said yes 5 times and was right 4 times.

Use it when

False alarms are costly, such as marking good email as spam.

~3 min

Recall

Of the real positives, how many were caught?

```
print(tp + fn)
print(round(tp / (tp + fn), 2))
print(round(mt.recall_score(actual, pred), 2))
```

Output

```
6
0.67
0.67
```

Formula

TP divided by TP + FN. There are 6 real positives and 4 were found.

Use it when

Misses are costly, such as screening for a disease. Recall is also called sensitivity.

~4 min

F1-Score

The harmonic mean punishes a weak side.

```
p, r = 0.9, 0.1
print((p + r) / 2)
print(round(2 * p * r / (p + r), 2))
print(round(mt.f1_score(actual, pred), 2))
```

Output

```
0.5
0.18
0.73
```

Plain average

Precision 0.9 and recall 0.1 average to 0.5, which looks fine.

Harmonic mean

F1 gives 0.18. It stays low unless both precision and recall are good.

~5 min

Precision Versus Recall

The cutoff trades one for the other.

```
import numpy as np
y = [0, 0, 1, 0, 1, 1, 0, 1, 1, 0]
scores = np.array([0.1, 0.3, 0.35, 0.4, 0.45,
                   0.6, 0.65, 0.7, 0.8, 0.9])
for t in (0.3, 0.5, 0.7):
    guess = (scores >= t).astype(int)
    p = mt.precision_score(y, guess)
    r = mt.recall_score(y, guess)
    print(t, round(p, 2), round(r, 2))
```

Output

```
0.3 0.56 1.0
0.5 0.6 0.6
0.7 0.67 0.4
```

Low cutoff

At 0.3 every real positive is found (recall 1.0), but many false alarms appear.

High cutoff

At 0.7 precision rises to 0.67, but recall drops to 0.4.

~4 min

Averages for Many Classes

One score per class, then combine.

```
a3 = [0, 0, 0, 1, 1, 1, 2, 2, 2, 2]
p3 = [0, 0, 1, 1, 1, 2, 2, 2, 2, 0]
for avg in ("macro", "micro", "weighted"):
    p = mt.precision_score(a3, p3, average=avg)
    print(avg, round(p, 2))
```

Output

```
macro 0.69
micro 0.7
weighted 0.7
```

macro

The plain average over classes. Every class counts equally.

micro and weighted

micro pools all calls together. weighted averages by class size.

~5 min

ROC Points

One point for every cutoff.

```
fpr, tpr, thr = mt.roc_curve(y, scores)
print(fpr.round(2).tolist())
print(tpr.round(2).tolist())
print(thr[1:].round(2).tolist())
```

Output

```
[0.0, 0.2, 0.2, 0.4, 0.4, 0.6, 0.6, 1.0]
[0.0, 0.0, 0.4, 0.4, 0.8, 0.8, 1.0, 1.0]
[0.9, 0.7, 0.65, 0.45, 0.4, 0.35, 0.1]
```

TPR and FPR

TPR is recall, TP / (TP + FN). FPR is FP / (FP + TN), the share of real negatives wrongly flagged.

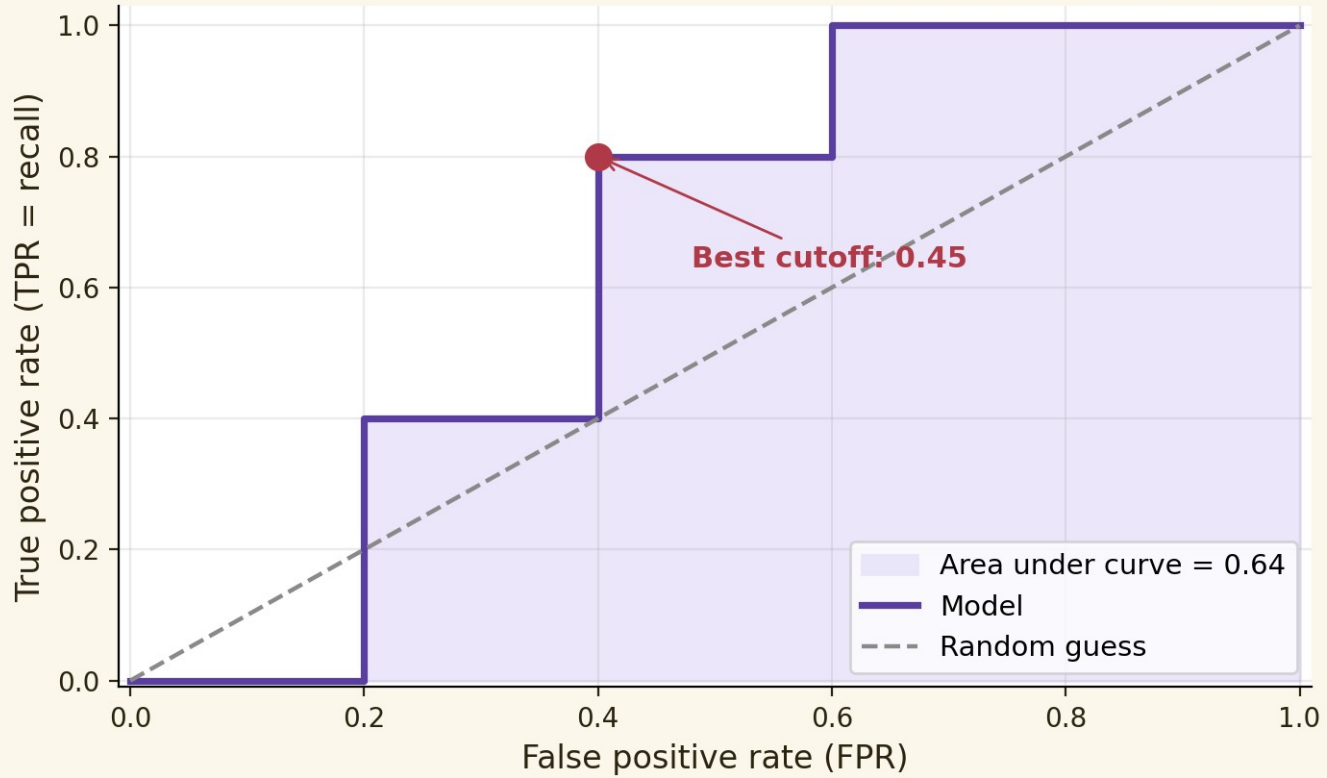
Thresholds

Each cutoff gives one point. A starting point (0, 0) is added with an infinite cutoff.

~4 min

Reading the ROC Curve

Plot TPR against FPR for every cutoff.



Up and left is better

A perfect model hugs the top-left corner. The dashed line is a random guess.

Moving along the curve

A lower cutoff moves up and right: more positives found, more false alarms.

~4 min

AUC: Area Under the Curve

One number for the whole curve.

```
print(round(mt.roc_auc_score(y, scores), 2))
print(mt.roc_auc_score(y, y))
print(mt.roc_auc_score(y, scores[::-1]))
print(mt.auc(fpr, tpr))
```

Output

```
0.64
1.0
0.36
0.64
```

Scale

1.0 is perfect and 0.5 is a random guess. Our scores give 0.64, only a little better than chance.

Below 0.5

Reversing the scores gives 0.36. The ranking is worse than random.

~4 min

Pick a Cutoff From the Curve

Largest gap between TPR and FPR.

```
j = tpr - fpr
best = j.argmax()
print(round(thr[best], 2), round(j[best], 2))
print(round(tpr[best], 1), round(fpr[best], 1))
```

Output

```
0.45 0.4
0.8 0.4
```

Youden's J

TPR minus FPR. The cutoff with the largest J is a balanced choice: 0.45.

At that point

It finds 80 percent of positives and flags 40 percent of negatives.

~3 min

Which Score Fits?

Pick the score that matches the cost.

Situation	Score
Classes are balanced	Accuracy
False alarms are costly	Precision
Misses are costly	Recall
Both errors matter	F1-score
Compare models over all cutoffs	ROC AUC
Very rare positive class	Precision and recall

Never report accuracy alone when one class is rare.

~3 min

Metrics and ROC Cheat Sheet

Match the task to the tool.

Task	Tool
Share correct	`accuracy_score`
Trust in a yes	`precision_score`
Positives found	`recall_score`
Balance of the two	`f1_score`
ROC points	`roc_curve(y, scores)`
Area under the curve	`roc_auc_score`

Scores for ROC need probabilities or decision scores, not hard 0 and 1 labels.

PYTHON FOR DATA · 40 MIN

Introduction to scikit-learn

Building, training and evaluating ML models in Python · Unit 4

Build
Pick and set up a model
`LogisticRegression`

Train
Learn from data
`fit`

Evaluate
Score on unseen data
`score`

Improve
Pipelines and tuning
`GridSearchCV`

~3 min

The scikit-learn Workflow

The same four steps for almost every model.

1

Load

Get features X and target y.

`load_iris`

2

Prepare

Split the data and scale it.

`train_test_split`

3

Train

Call fit on the training data.

`model.fit`

4

Evaluate

Score on data the model never saw.

`model.score`

Every scikit-learn model follows the same fit, predict and score pattern.

~3 min

Three Kinds of Objects

One consistent interface.

Object	Key methods	Example
Estimator	`fit, predict, score`	`LogisticRegression`
Transformer	`fit, transform`	`StandardScaler`
Pipeline	`fit, predict, score`	`make_pipeline`
Search	`fit, best_params_`	`GridSearchCV`

Learn one pattern, and you can swap in any other model.

~4 min

Loading Data

Built-in data sets for practice.

```
from sklearn.datasets import load_iris
iris = load_iris()
X, y = iris.data, iris.target
print(X.shape, y.shape)
print(iris.feature_names[:2])
print(iris.target_names.tolist())
```

Output

```
(150, 4) (150,)
['sepal length (cm)', 'sepal width (cm)']
['setosa', 'versicolor', 'virginica']
```

X and y

X is a table of 150 flowers and 4 measurements. y holds the species as 0, 1 or 2.

Names

feature_names and target_names say what the columns and numbers mean.

~4 min

Splitting the Data

Keep a test set locked away.

```
import numpy as np
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=0)
print(X_tr.shape, X_te.shape)
print(np.bincount(y_te).tolist())
```

Output

```
(105, 4) (45, 4)
[15, 15, 15]
```

70 and 30

105 rows train the model. 45 rows are kept for testing.

stratify

Each species appears equally in the test set: 15 of each.

~4 min

Training a Model

Create it, then fit it.

```
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier(max_depth=3,
                              random_state=0)
tree.fit(X_tr, y_tr)
print(tree.predict(X_te[:5]).tolist())
print(round(tree.score(X_te, y_te), 3))
```

Output

```
[2, 2, 0, 0, 1]
0.956
```

fit

Learns from the training rows. Settings such as max_depth are chosen before fitting.

predict and score

predict returns classes. score returns accuracy on the test rows: 0.956.

~4 min

Evaluating a Model

Look beyond a single number.

```
from sklearn import metrics as mt
pred = tree.predict(X_te)
print(round(mt.accuracy_score(y_te, pred), 3))
print(mt.confusion_matrix(y_te, pred).tolist())
rep = mt.classification_report(
    y_te, pred, output_dict=True)
print(round(rep["macro avg"]["f1-score"], 3))
```

Output

```
0.956
[[15, 0, 0], [0, 15, 0], [0, 2, 13]]
0.955
```

Confusion matrix

The tree mixes up two flowers: 2 virginica were called versicolor.

macro F1

The average F1 over all three classes is 0.955.

~4 min

Trying Other Models

Same code, different model.

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(random_state=0)
models = [LogisticRegression(),
KNeighborsClassifier(), rf]
for m in models:
    m.fit(X_tr, y_tr)
    acc = round(m.score(X_te, y_te), 3)
    print(type(m).__name__, acc)
```

Output

```
LogisticRegression 1.0
KNeighborsClassifier 1.0
RandomForestClassifier 0.978
```

One interface

Every model uses fit and score, so a loop can compare them.

Be careful

45 test rows is small. Confirm the winner with cross-validation.

~4 min

Scaling Features

A transformer learns, then applies.

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)
X_te_s = sc.transform(X_te)
print(abs(X_tr_s.mean(axis=0)).round(2).tolist())
print(X_tr_s.std(axis=0).round(2).tolist())
```

Output

```
[0.0, 0.0, 0.0, 0.0]
[1.0, 1.0, 1.0, 1.0]
```

fit_transform

On training data, learn the mean and spread, then apply them.

transform only

On test data, reuse the training numbers. Never fit on the test set.

~4 min

Pipelines

Chain the steps into one model.

```
from sklearn.pipeline import make_pipeline
scaler = StandardScaler()
model = LogisticRegression(max_iter=1000)
pipe = make_pipeline(scaler, model)
pipe.fit(X_tr, y_tr)
print(round(pipe.score(X_te, y_te), 3))
print(pipe.steps[0][0])
```

Output

```
0.978
standardscaler
```

One object

A pipeline scales, then models. fit and predict run every step in order.

Fewer mistakes

The scaler is fitted only on training data, so nothing leaks from the test set.

~4 min

Cross-Validation

Score on several splits.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(pipe, X, y, cv=5)
print(scores.round(2).tolist())
print(round(scores.mean(), 3))
```

Output

```
[0.97, 1.0, 0.93, 0.9, 1.0]
0.96
```

Five scores

The data is cut into 5 folds. Each fold takes a turn as the test set.

Mean

0.96 is a steadier estimate than the single 0.978 from one split.

~5 min

Tuning Settings

GridSearchCV tries the options.

```
from sklearn.model_selection import GridSearchCV
grid = {"logisticregression__C": [0.1, 1, 10]}
gs = GridSearchCV(pipe, grid, cv=5)
gs.fit(X_tr, y_tr)
print(gs.best_params_)
print(round(gs.best_score_, 3))
print(round(gs.score(X_te, y_te), 3))
```

Output

```
{'logisticregression__C': 10}
0.962
1.0
```

Step name and setting

logisticregression__C means the setting C of the step named logisticregression.

Test once

Tune on the training data only. Use the test set once, at the end.

~4 min

A Regression Model

Predict a number instead of a class.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
Xd, yd = load_diabetes(return_X_y=True)
Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(
    Xd, yd, test_size=0.2, random_state=0)
reg = LinearRegression().fit(Xd_tr, yd_tr)
print(round(reg.score(Xd_te, yd_te), 3))
print(round(mt.mean_absolute_error(
    yd_te, reg.predict(Xd_te)), 1))
```

Output

```
0.332
46.2
```

R squared

For regression, score is R squared. 0.332 means the model explains a third of the variation.

MAE

The average miss is 46.2 units of disease progression. Try other models to do better.

~4 min

Saving a Model

Reuse it without retraining.

```
import joblib
joblib.dump(pipe, "iris_model.joblib")
loaded = joblib.load("iris_model.joblib")
print(loaded.predict(X_te[:3]).tolist())
print(pipe.predict(X_te[:3]).tolist())
```

Output

```
[2, 2, 0]
[2, 2, 0]
```

dump and load

Writes the fitted pipeline to a file and reads it back later.

Trust the file

Only load model files you created. Loading a file can run code.

~3 min

scikit-learn Cheat Sheet

Match the task to the tool.

Task	Tool
Load practice data	`load_iris, load_diabetes`
Split the data	`train_test_split`
Train and predict	`fit, predict, score`
Scale features	`StandardScaler, make_pipeline`
Score on many splits	`cross_val_score`
Tune and save	`GridSearchCV, joblib`

Split first, fit on training data only, and test once at the end.